

42

Platform As-Is Report, Gap Analysis & Product Requirements Document

Document Version: 1.0

Date: July 2026

Checkpoint: debd909

Classification: Technical – Internal

Table of Contents

Executive Summary

Part I — As-Is Report

1. Architecture Overview
2. Data Model
3. Game Engine
4. Frontend Architecture
5. Backup & Safety System
6. Current Capabilities
7. Current Limitations

Part II — Gap Analysis

8. Reference Visual Target
9. Gap Matrix
10. Critical Gaps Summary

Part III — Product Requirements Document

11. Vision & Objectives
12. Functional Requirements
13. Non-Functional Requirements
14. Technical Architecture Evolution
15. Phased Roadmap
16. Success Metrics
17. Risks & Mitigations
18. Appendix
19. Document Control

42 — Platform As-Is Report, Gap Analysis & Product Requirements Document

Document Version: 1.0

Date: July 2026

Checkpoint: `debd909` (sole canonical version)

Classification: Technical — Internal

Executive Summary

42 is a gamified, real-time strategy command interface simulating a decentralized compute network where three factions — **FANG**, **HAMMER**, and **RESOLUTE** — wage live wargames across a real 1:1 satellite Earth globe. The platform combines a Next.js 16 frontend, a socket.io authoritative game engine, an LLM-powered tactical co-pilot (ARIA), and a real Esri World Imagery base map styled in a monochrome tactical aesthetic.

This document provides a complete technical inventory of the platform's current state (As-Is), a structured gap analysis against the product vision (a full data dashboard with drone-attack visualization at the fidelity of professional surveillance platforms), and a phased Product Requirements Document (PRD) defining the work required to close the gaps and scale to millions of users.

The platform is currently at a **functional MVP** stage: the core game loop (territory control, missions, outposts, AI briefings) is operational and visually grounded on a real Earth globe, but several layers — unit-level telemetry, dashboard-grade data visualization, and production scaling infrastructure — remain to be built.

PART I — AS-IS REPORT

1. Architecture Overview

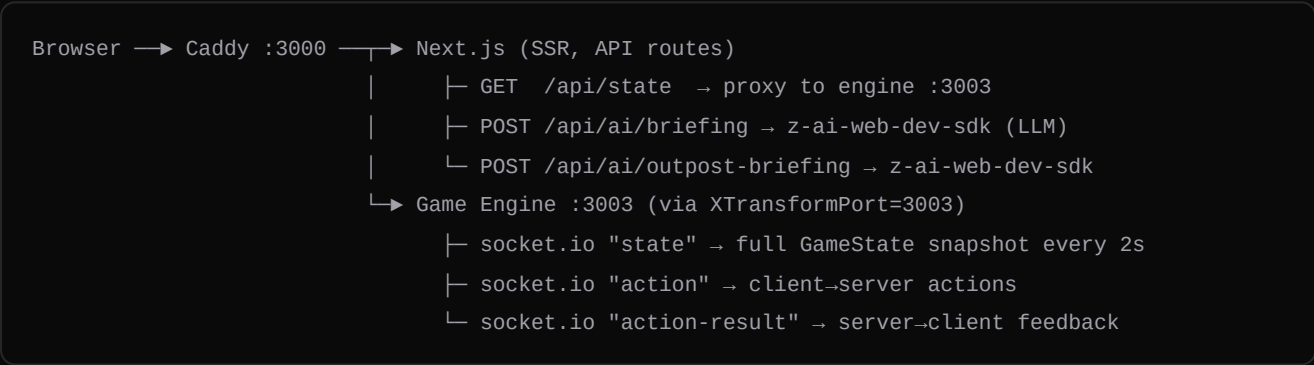
1.1 System Topology

The platform consists of three runtime processes plus a persistent storage layer:

COMPONENT	PORT	ROLE	PERSISTENCE
Next.js App (frontend + API)	3000	React UI, REST API routes, SSR gateway	Rootfs (regenerable)
Game Engine (socket.io)	3003	Authoritative in-memory GameState, game loop, client actions	Rootfs (in-memory)
Caddy Gateway	80/443	Single external port; <code>XTransformPort</code> query routing	Rootfs (regenerable)
SQLite (Prisma)	File	Operative profile + war log persistence	Rootfs (tracked in git)

All three processes run concurrently. The Caddy gateway exposes a single external port and routes API/WebSocket traffic to the correct internal port via the `?XTransformPort=<port>` query parameter. This is a hard constraint of the sandbox environment.

1.2 Request Flow



The frontend uses a **dual-channel** data strategy: an initial REST fetch (`/api/state`) for the first snapshot, then a persistent socket.io connection for real-time 2-second ticks.

1.3 Technology Stack

LAYER	TECHNOLOGY	VERSION	NOTES
Framework	Next.js (App Router)	16.1.1	Single route <code>/</code> only
Language	TypeScript	5.x	Strict typing throughout
Styling	Tailwind CSS	4.x	+ shadcn/ui (New York)
State (client)	Zustand	5.0.6	<code>useCommand</code> store
State (server)	In-memory (engine)	—	GameState object
Real-time	socket.io	4.8.3	Client + server
Map	MapLibre GL JS	5.24.0	3D globe projection
Map data	world-atlas	2.0.2	countries-10m (1:10M, 255 countries)
Map imagery	Esri World Imagery	—	Raster satellite tiles, no API key
Database	Prisma + SQLite	6.11.1	Operative + WarLog models
AI	z-ai-web-dev-sdk	0.0.18	LLM (server-only)
Animation	Framer Motion	12.23.2	UI transitions
Charts	Recharts	2.15.4	Available but underutilized
Icons	Lucide React	0.525.0	—
Auth	NextAuth.js	4.24.11	Available, not yet wired
Validation	Zod	4.0.2	—

2. Data Model

2.1 Core Domain Types

The shared type contract lives in `src/lib/types.ts` and is the source of truth between the engine, API, and client.

FactionId — `"FANG" | "HAMMER" | "RESOLUTE"` (3 factions)

Outpost — A node on the globe controlled by a faction:

- `id`, `name`, `type` (`FULL` | `TACTICAL` | `SAFEHOUSE`), `faction`
- `lat`, `lng` (real geographic coordinates)
- `level`, `health`, `maxHealth`, `compute` (TFLOPS), `uptime` (seconds), `buildPoints`
- `status` (`ONLINE` | `DEGRADED` | `OFFLINE` | `UNDER_ATTACK`)
- `establishedAt` (epoch ms)

Mission — An in-progress operation between two outposts:

- `type` : `DRONE_STRIKE` | `CYBER_ATTACK` | `ESPIONAGE` | `RECON` | `BUILD` | `DEFEND`
- `status` : `QUEUED` | `ACTIVE` | `COMPLETE` | `FAILED`
- `sourceId` , `targetId` (outpost IDs), `faction`
- `progress` (0–100), `eta` (seconds), `startedAt` , `label`

Territory — A named geographic region factions fight to control:

- `id` , `name` , `polygon` ([`lng`,`lat`] closed ring), `center`
- `controller` (`FactionId` | null), `control` (0–100 dominance)
- `influence` (per-faction influence sum)

ActivityPing — Transient live-action point ("millions of actions" visualization):

- `lat` , `lng` , `type` (`STRIKE` | `BUILD` | `DEPLOY` | `SCAN` | `BREACH` | `TRAFFIC`)
- `faction` , `intensity` (0–1), `bornAt` (epoch ms)

Faction — Aggregate faction stats: `strength` , `compute` , `territories` , `outposts` , `threat`

Operative — The player: `codename` , `tier` (`BASIC` | `ELITE` | `ARCHON`), `faction` , `authority`

GameState — The authoritative snapshot: `tick` , `sol` , `clock` , `factions` , `outposts` , `missions` , `events` , `threatLevel` , `operative` , `briefing` , `territories` , `activityPings` , `networkLoad` , `totalActions`

2.2 Currency System

- **VOTC** — Network-universal currency
- **Faction tokens** — Each faction mints its own token (FANG, HAMMER, RESOLUTE) for intra-faction upgrades and tribute

2.3 Database Schema (Prisma/SQLite)

The live game state is in-memory for speed. SQLite persists only:

- **Operative** — codename, tier, faction, authority, victories, defeats
- **WarLog** — resolved mission outcomes (sol, operative, faction, outcome, summary)

2.4 Territory Definitions

12 named geographic regions are hardcoded in the engine:

TERRITORY	CENTER	CONTROLLER LOGIC
North America	[-100, 45]	Aggregate outpost influence (level + health + proximity)
South America	[-65, -20]	Recomputed every tick
Europe	[15, 50]	Controller flips when leading faction changes
Africa	[20, 0]	Event pushed on flip
Middle East	[47, 28]	—
Central Asia	[65, 47]	—
South Asia	[75, 22]	—
East Asia	[115, 38]	—
Southeast Asia	[115, 5]	—
Oceania	[135, -25]	—
Arctic	[-30, 80]	—
Atlantic	[-30, 50]	—

2.5 Seeded Outposts

16 outposts are seeded across real-world cities at boot:

FACTION	OUTPOSTS	CITIES
FANG	5	New York, London, Tokyo, Dubai, Sydney
HAMMER	6	San Francisco, Stockholm, Singapore, Buenos Aires, Rio, Moscow
RESOLUTE	5	Hong Kong, Istanbul, Lagos, Cairo, Reykjavik

3. Game Engine (Port 3003)

3.1 Architecture

The engine is a standalone Bun mini-service (`mini-services/game-engine/index.ts` , 895 lines) running socket.io on port 3003. It holds the authoritative `GameState` in memory and runs a game loop every 2 seconds (`TICK_MS = 2000`).

3.2 Game Loop

Each tick performs:

1. **Uptime accrual** — Each outpost's `uptime` increases; `buildPoints` accrue from uptime

2. **Mission progression** — Active missions advance `progress`, decrement `eta`; on completion, effects are applied (damage, build points, threat changes)
3. **Territory recalculation** — Per-territory influence is recomputed from outpost level + health + proximity to centroid; controllers flip when the leading faction changes
4. **Faction recalculation** — Aggregate `strength`, `compute`, `outposts`, `territories`, `threat` per faction
5. **Activity ping synthesis** — 2–4 new pings spawned per tick near random outposts, weighted by type (TRAFFIC 40%, BUILD 20%, SCAN 15%, DEPLOY 10%, BREACH 8%, STRIKE 7%)
6. **Event generation** — Faction AI launches occasional missions; events pushed for flips, strikes, breaches
7. **Broadcast** — Full `GameState` snapshot emitted to all connected clients

3.3 Client Actions (Socket Protocol)

Client → Server (`socket.emit("action", ...)`):

- `launch-mission` — `{ missionType, sourceId, targetId }`
- `place-outpost` — `{ type, lat, lng, name? }` (user clicks globe to deploy)
- `upgrade-outpost` — `{ id }`
- `request-briefing` — triggers AI briefing generation

Server → Client:

- `state` — full `GameState` snapshot (every 2s)
- `action-result` — `{ ok, error?, note? }` feedback on submitted actions

3.4 Scaling Architecture (Current vs Target)

The engine code documents the target scaling architecture explicitly:

- **Current:** Single-process, in-memory, full-state snapshot every 2s (~16 outposts + ~80 pings + 12 territories — small enough for simplicity)
- **Target (millions of users):** (a) shard state by region (each territory = independent actor), (b) emit delta updates instead of full snapshots, (c) batch action ingress at an edge gateway

4. Frontend Architecture

4.1 Component Hierarchy

The UI is a single-page floating-HUD architecture over a full-screen globe:


```

<CommandDeck>
  └─ <BootScreen />           ─ "ESTABLISH UPLINK" gate (audio resume + socket init)
  └─ <WorldMap />             ─ Full-screen MapLibre 3D globe (all graphics localized)
  └─ <CommandBar />           ─ Top floating bar (brand, clock, threat, standings, profile)
    └─ <BrandArea />          ─ Logo + system name
    └─ <ClockArea />          ─ Sol cycle + real-time clock
    └─ <ThreatArea />         ─ Global threat level (GREEN/AMBER/RED/BLACK)
    └─ <StandingsArea />      ─ 3-faction strength leaderboard
    └─ <StatsArea />          ─ Active nodes, network load, total actions
    └─ <ProfileArea />        ─ Operative codename + tier + authority
  └─ <NavRail />              ─ Left vertical nav (8 views: MAP/FEED/STRIKE/QUEUE/AI/INTEL/DEPLO
  └─ <LeftPanel />            ─ Context panel (mission launcher, feed, AI briefing)
  └─ <RightPanel />           ─ Outpost detail / list
  └─ <OutpostDetailCard />    ─ Selected outpost details + actions
  └─ <StatusBar />            ─ Bottom bar (connection, tick, coords)
  └─ <Toaster />              ─ Toast notifications

```

4.2 State Management

The `useCommand` Zustand store (`src/stores/command.ts`) holds:

- `state: GameState | null` — live game state from socket
- `connected: boolean` — socket connection status
- `briefing: Briefing | null` + `briefingLoading` — AI briefing
- `selectedOutpostId`, `pendingMission`, `placementMode` — UI interaction state
- `socket: Socket | null` — singleton socket.io connection

The socket connects via `io("/", { query: { XTransformPort: "3003" }, transports: ["polling", "websocket"] })`.

4.3 Map/Globe Implementation

The globe (`src/components/command/world-map.tsx`, ~1090 lines) is the visual centerpiece. Key implementation details:

Base Layer — Real 1:1 Earth:

- **Esri World Imagery** raster tiles
(`server.arcgisonline.com/.../World_Imagery/MapServer/tile/{z}/{y}/{x}`)
- No API key, no server, no download — lightweight raster tiles
- Monochrome-tactical paint: `raster-saturation: -0.85` (near-grayscale), `raster-brightness-max: 0.85`, `raster-contrast: 0.15`

Vector Overlay:

- **Natural Earth 10m** country data (255 countries, 544,898 coordinate points — 27× more detail than 110m)
- Zoom-responsive coastline width (0.5px → 2.4px across zoom 0 → 8)
- Faint white landmass fill + dark ocean tint

Projection:

- `projection: { type: "globe" }` set in the style spec (guaranteed active from first render, not a post-load hack)

Gameplay Layers (22 total, bottom → top):

1. `satellite-base` — real Earth imagery
2. `ocean-fill` — dark ocean tint
3. `graticule` — lat/lng grid lines (every 15°)
4. `countries-fill` / `countries-line` — vector coastlines + borders
5. `territory-fill` / `territory-line` / `territory-label` — 12 territory control polygons (fill opacity = control %)
6. `halos-fill` — per-outpost influence circles
7. `activity-pings` — sonar/heat layer (expanding rings, aged out by `bornAt`)
8. `vectors-agg` / `vectors-pass` — mission attack arcs (great-circle paths, aggressive vs passive)
9. `vectors-agg-impact` — pulsing impact points at active strike targets
10. `progress-heads` — moving dots along active mission arcs
11. `outpost-cluster` / `outpost-safehouse-aura` / `outpost-pulse` / `outpost-health-ring` / `outpost-shape` — clustered outpost markers with faction sprites (hex/diamond/square), health rings, under-attack pulses, safehouse fortified auras

Performance Optimizations:

- Outpost clustering (`clusterRadius 32`, `clusterMaxZoom 5`) — collapses dense regions below zoom 5
- Visible-hemisphere culling — pings outside ~100° of map center are dropped
- Active-mission-only arc rendering — arcs built only for ACTIVE/COMPLETE missions
- Throttled animation loops (10–12fps via `setInterval`, not `rAF`) for ping/progress motion

Interaction:

- Click outpost → select + open detail card
- Click globe in placement mode → deploy new outpost at real lat/lng
- Drag-rotate, scroll-zoom, touch-zoom-rotate, double-click-zoom all enabled

4.4 AI Co-Pilot (ARIA)

Two LLM-powered API routes use `z-ai-web-dev-sdk` (server-only):

POST /api/ai/briefing — Global tactical briefing:

- Sends compacted GameState (factions, outposts, missions, events) to the LLM
- System prompt: "You are ARIA, the AI operations co-pilot..."
- Returns strict JSON: `{ summary, threatAssessment, recommendations[] }`
- Each recommendation: `{ id, title, rationale, action: "launch:TYPE:src:tgt", confidence }`
- Fallback: deterministic heuristic briefing if LLM fails

POST /api/ai/outpost-briefing — Per-outpost situational assessment:

- Returns `OutpostBrief` : `{ assessment, recommendation, priority, confidence, votcAtStake, token }`

4.5 Audio

`useSfx` hook provides:

- Ticking sound during boot (170ms interval)
- Key/confirm/deny sounds for actions
- `AudioContext` resume on user gesture (browser autoplay policy)

5. Backup & Safety System

5.1 Four-Layer Backup Architecture

The platform has a verified 4-layer backup system to prevent progress loss:

LAYER	LOCATION	PERSISTENCE	SIZE	RESTORE COMMAND
Bare repo	<code>/home/z/42-backup.git</code>	Rootfs	—	<code>./RESTORE.sh bare</code>
Git bundles (×3)	<code>upload/</code> , <code>/home/sync/42-backups/</code> , <code>/tmp/my-project/</code>	OSS cloud ×2 + PolarFS	7.7 MB	<code>./RESTORE.sh bundle</code>
Tarballs (×3)	Same 3 locations	OSS cloud ×2 + PolarFS	7.8 MB	<code>./RESTORE.sh tarball</code>
Working .git	<code>/home/z/my-project/</code>	Rootfs	—	Live

All 3 persistent storage systems survive environment resets:

- `upload/` — OSS cloud mount
- `/home/sync/` — OSS cloud mount (second independent OSS)
- `/tmp/my-project/` — PolarFS mount

5.2 Git Safety Hooks

- `.githooks/pre-reset` — Auto-creates a timestamped bundle in `upload/` before any `git reset --hard` (keeps 5 most recent)
- `.githooks/pre-commit` — Refreshes `upload/42-base.bundle` + `/home/sync/42-backups/42-base.bundle` before every commit

Hooks are activated via `git config core.hooksPath .githooks`.

5.3 Restore Verification

All 3 restore paths were verified end-to-end against checkpoint `debd909`:

- Bare repo clone → HEAD `debd909` , 142 files
- Bundle clone (`git clone -b main`) → HEAD `debd909` , 142 files, satellite + 10m data confirmed
- Tarball extract → 142 files, all critical files present

5.4 Current Checkpoint

- **Commit:** `debd9092edf0471f506d38483100bd6dee77e13a`
- **Tag:** `base` (annotated, points to checkpoint)
- **Files:** 142 tracked
- **History:** Single parentless commit (orphan-branch squash — no prior history, clean checkpoint)

6. Current Capabilities (What Works)

6.1 Functional

- Real-time 3D globe with actual satellite Earth imagery (Esri World Imagery)
- 3 factions with 16 seeded outposts at real global cities
- 12 territory control zones with live influence recalculation and visual flips
- 6 mission types (DRONE_STRIKE, CYBER_ATTACK, ESPIONAGE, RECON, BUILD, DEFEND) with progress arcs
- Activity ping "sonar" layer simulating planet-scale activity (~80 concurrent pings)
- Safehouse node placement (user clicks globe to deploy at real coordinates)
- Outpost upgrade system (build points → level/health)
- AI tactical briefings (LLM-generated, with heuristic fallback)
- Per-outpost AI situational assessments
- Boot screen with audio gate + socket initialization
- Floating HUD: command bar, nav rail, left/right panels, status bar, outpost detail card
- Toast feedback on all actions (confirm/deny sounds)
- Keyboard navigation (1–8 for views, ESC to cancel)
- Network load + total actions counters (simulated millions-scale metrics)

6.2 Visual

- Monochrome black-and-white tactical aesthetic
- Desaturated satellite base (near-grayscale, darkened)
- White vector coastlines + borders (zoom-responsive)
- Faction shape markers (hex/diamond/square sprites)
- Health rings, under-attack pulses, safehouse fortified auras
- Great-circle attack arcs with animated progress heads
- Expanding sonar rings for activity pings
- Territory fill opacity driven by control percentage

6.3 Infrastructure

- 4-layer backup system (all verified restorable)
- Git safety hooks (auto-backup before destructive ops)
- Single external port via Caddy gateway
- Both processes (Next.js + engine) running concurrently
- Lint clean, dev server stable

7. Current Limitations (What Doesn't Work Yet)

7.1 Scale

- Single-process in-memory engine (no sharding, no delta updates)
- Full-state snapshot every 2s (bandwidth scales $O(n)$ with state size, not $O(\text{changes})$)
- No authentication — single operative, no multi-user
- No persistent game state across engine restarts (in-memory only)

7.2 Data

- All outposts/missions are synthetic (seeded + AI-driven, not real user actions)
- Activity pings are synthesized by the engine, not from real telemetry
- No real drone/unit telemetry
- No historical data (no time-series storage, no replay)

7.3 Visualization

- No unit-level detail view (no wireframe models, no per-unit telemetry dashboard)
- No charts/graphs in the UI (Recharts installed but unused)
- No data tables (no sortable/filterable unit lists)
- No timeline/feed visualization (events exist but no dedicated timeline panel)
- No heatmap density visualization (pings are discrete points, not aggregated heat)

7.4 Interaction

- No drag-and-drop mission planning
- No multi-select for batch operations
- No replay/scrubbing of historical state
- No export/report generation from the UI

PART II — GAP ANALYSIS

8. Reference Visual Target

The product vision references a professional surveillance-platform aesthetic (exemplified by the SurveilTrack drone-dashboard reference image). The key visual fidelity characteristics of that target:

8.1 Reference Layout

- **Header bar:** Logo, navigation tabs (BRIEF / UNIT MAP / SETUP), contextual info (location, time)
- **Left sidebar:** Vertical icon menu (grid, drone, shield, warning, lock, users, headset, profile, settings)
- **Main area split:** Left panel (unit details, wireframe model, metrics) + right panel (3D map)
- **Bottom bar:** Tabbed navigation (Unit list / Statistics / Performances / Overview / Messages) + horizontal scrollable unit list with status indicators
- **Map controls:** Zoom +/- and full-screen buttons on the map edge

8.2 Reference Map

- High-fidelity 3D satellite terrain (city-level detail: buildings, waterways, streets)
- Dark monochromatic scheme (buildings near-black, waterways dark gray, streets thin white)
- Discrete unit markers (small white squares with labels) positioned above buildings
- Status indicators (green = active, red = inactive, gray = disabled)
- Subtle dashed connection lines between units and detail panels

8.3 Reference UI Chrome

- Black panels with thin white borders, high contrast
- Wireframe 3D models for unit details
- Progress bars for metrics (power, signal)
- Horizontal tabular unit lists with status dots
- Stats in white text with uppercase labels

8.4 Reference Color & Typography

- **Background:** `#1A1A1A` (panels), `#222222` (map terrain)
- **Text:** `#FFFFFF` (all labels/metrics)
- **Status accents:** `#00FF00` (active), `#FF0000` (inactive), `#808080` (disabled), `#FFA500` (moderate)
- **Typography:** Bold uppercase sans-serif for headers (~18px, weight 700), medium uppercase for nav (~14px, weight 500), regular for metrics (~14px, weight 400)

9. Gap Matrix

CAPABILITY	REFERENCE TARGET	CURRENT STATE	GAP SEVERITY
Base map	3D satellite terrain (city-level)	Esri satellite globe (continent-level zoom)	LOW — data source is correct; needs higher zoom + terrain
Map projection	3D globe + city-level 3D	3D globe (globe projection active)	NONE — projection matches
Unit markers	Discrete labeled squares with status dots	Faction shape sprites (hex/diamond/square) with health rings	MEDIUM — need unit-level markers, not just outpost-level
Unit detail panel	Wireframe 3D model + metrics (power, session, signal)	Outpost detail card (health, compute, build points)	MEDIUM — needs richer telemetry + visual model
Unit list	Horizontal scrollable list with status dots (active/inactive/disabled)	Right panel outpost list	MEDIUM — needs status taxonomy + horizontal scroll pattern
Bottom tabbed bar	Unit list / Statistics / Performances / Overview / Messages	NavRail (8 vertical views)	LOW — navigation exists, structure differs
Charts/statistics	Performance charts, statistics tab	None (Recharts installed, unused)	HIGH — no data visualization
Attack vectors	Connection lines between units	Great-circle mission arcs (animated)	NONE — exceeds reference
Territory zones	Not in reference	12 territory polygons with control %	EXCEEDS reference
Activity heat	Not in reference	Activity ping sonar layer	EXCEEDS reference
Threat level	Not in reference	Global threat level (GREEN/AMBER/RED/BLACK)	EXCEEDS reference
AI co-pilot	Not in reference	ARIA LLM briefings + per-outpost assessments	EXCEEDS reference
Drone attacks	Core feature (drone surveillance)	DRONE_STRIKE mission type (arc + impact pulse)	MEDIUM — visual is good, no drone-level telemetry
Real-time data	Live unit status	2s tick snapshots	LOW — cadence is good
Color palette	Green/red/gray status dots	Pure monochrome (white-on-black)	MEDIUM — status colors would aid scannability

Typography	Bold uppercase sans-serif	Monospace tactical font	LOW — current is thematically appropriate
Multi-user scale	Implied (fleet of units)	Single operative, 16 outposts	HIGH — no authentication, no real users
Historical data	Performance tab implies history	None (in-memory, no time-series)	HIGH — no replay, no trends

10. Critical Gaps Summary

10.1 Gap A: No Data Dashboard Layer (HIGH)

The platform has a beautiful globe but lacks the dashboard-grade data visualization tools shown in the reference. There are no charts, no sortable tables, no statistics panels, no performance trends. Recharts is installed but completely unused. The reference's "Statistics / Performances" tabs have no equivalent.

10.2 Gap B: No Unit-Level Telemetry (HIGH)

The platform operates at the **outpost** level (16 nodes), but the reference operates at the **unit** level (individual drones with power, signal, session time). There is no concept of individual units within an outpost, no per-unit telemetry, no unit status taxonomy (active/inactive/disabled).

10.3 Gap C: No Multi-User / Authentication (HIGH)

NextAuth.js is installed but not wired. The platform supports a single operative with no login. Scaling to "millions of users taking millions of actions" requires authentication, per-user state, and action attribution.

10.4 Gap D: No Historical Data / Replay (MEDIUM)

All state is in-memory with no time-series storage. There is no way to review past missions, track performance trends, or replay territory changes over time. The WarLog table exists but is not populated.

10.5 Gap E: No Real Telemetry Source (MEDIUM)

All activity is synthesized by the engine. Activity pings, mission outcomes, and faction AI are all simulated. There is no ingestion of real user actions, no real compute metrics, no real network load data.

10.6 Gap F: Status Color Taxonomy Missing (MEDIUM)

The reference uses green/red/gray for instant scannability of unit status. The platform is pure monochrome. While thematically consistent, this reduces situational awareness speed for large unit counts.

10.7 Gap G: Production Scaling Infrastructure (MEDIUM)

The engine is single-process. The code documents the target architecture (regional sharding, delta updates, edge action batching) but none is implemented. Bandwidth scales $O(n)$ with state size.

PART III — PRODUCT REQUIREMENTS DOCUMENT

11. Vision & Objectives

11.1 Product Vision

42 is a real-time strategy command platform where millions of users command factions in a persistent global wargame, fighting for territory, launching drone strikes, deploying safehouses, and managing compute infrastructure — all visualized on a real 1:1 Earth globe with dashboard-grade data tools and an AI tactical co-pilot.

11.2 Design Principles

1. **Real geography, real stakes** — Every action occurs at a real lat/lng on actual satellite Earth
2. **Dashboard clarity** — Data must be scannable in <3 seconds (tables, charts, status colors)
3. **Cinematic tactical aesthetic** — Monochrome dark theme, monospace typography, military-grade UI
4. **Planet-scale feel** — Activity pings and network-load counters convey millions of actions
5. **AI-augmented decisions** — ARIA co-pilot provides actionable recommendations, not just data
6. **Never lose progress** — 4-layer backup system is non-negotiable

11.3 Target Personas

PERSONA	DESCRIPTION	PRIMARY NEED
Commander	Faction leader managing multiple outposts	Strategic overview + AI briefings
Operative	Individual user running missions	Unit-level control + mission queue
Analyst	Observer reviewing performance	Historical data + statistics + trends
Recruit	New user onboarding	Clear deployment flow + safehouse placement

12. Functional Requirements

Phase 1: Dashboard & Telemetry (Closes Gaps A, B, F)

FR-1.1: Unit-Level Telemetry Model

- Add `Unit` type to the data model: `{ id, outpostRef, type, status, power, signal, sessionTime, lastAction }`
- Each outpost contains 1–N units (drones, workers, scanners)

- Unit status taxonomy: `ACTIVE` | `INACTIVE` | `DISABLED` | `ENGAGED`
- Engine synthesizes per-unit telemetry each tick

FR-1.2: Unit Detail Panel

- Wireframe-style unit visualization (SVG/CSS, not 3D — keep it lightweight)
- Real-time metrics: power %, signal strength, session duration, action count
- Status indicator (colored dot: green/red/gray/orange)
- Action history (last 5 actions for this unit)

FR-1.3: Unit List (Horizontal Scrollable)

- Bottom-bar horizontal scrollable list of all units for the operative's faction
- Each card: unit ID, status dot, power %, current task
- Click → opens unit detail panel
- Filter by status (all / active / inactive / disabled)

FR-1.4: Statistics Dashboard

- New NavRail view: `STATS` (replaces or augments existing view)
- Faction strength over time (line chart)
- Territory control breakdown (stacked bar)
- Mission success rate (donut)
- Top operatives leaderboard (table)
- Network load trend (area chart)

FR-1.5: Performance Tab

- Per-operative performance metrics
- Missions launched / succeeded / failed
- Compute contributed
- Territory captures
- Kill/death ratio (outpost losses)

FR-1.6: Status Color System

- Introduce a controlled color palette for status only (not for faction identity — factions stay monochrome via shape):
- `ACTIVE` : `#22c55e` (green)
- `INACTIVE` : `#ef4444` (red)
- `DISABLED` : `#71717a` (gray)
- `ENGAGED` : `#f59e0b` (amber)
- Apply to unit status dots, outpost status badges, and event severity icons

Phase 2: Multi-User & Persistence (Closes Gaps C, D)

FR-2.1: Authentication

- Wire NextAuth.js with credential provider (email + password)
- `Operative` model becomes the user profile

- Session → operative mapping
- Guest mode preserved (boot screen → "ESTABLISH UPLINK" → demo operative)

FR-2.2: Per-User State

- Each authenticated operative has their own outpost subset
- Actions attributed to operative ID
- Operative-scoped mission queue
- Operative-scoped build points + VOTC balance

FR-2.3: Time-Series Storage

- New Prisma model: `StateSnapshot` (sol, gameState JSON, createdAt)
- Engine writes a snapshot every 60 seconds (1/min = 1440/day)
- Retention: 7 days at 1-min granularity, then downsample to 1-hour for 90 days

FR-2.4: Replay System

- New NavRail view: `REPLAY`
- Timeline scrubber (sol-based)
- Play/pause/step controls
- Globe renders historical state at selected timestamp
- Territory flips + mission outcomes replayed

FR-2.5: WarLog Population

- Engine writes to `warLog` on every mission completion
- Outcome, summary, operative, faction, sol
- Queryable in the Stats dashboard

Phase 3: Real Telemetry Ingestion (Closes Gap E)

FR-3.1: Action Ingestion API

- `POST /api/actions` — operatives submit actions via REST (not just socket)
- Action queue with rate limiting (per-operative)
- Action validation (source ownership, target validity, resource cost)

FR-3.2: Real Compute Metrics

- If the platform connects to real compute nodes (browser-based workers via TACTICAL outposts), ingest real uptime/compute
- Worker heartbeat protocol (WebWorker → engine)
- Real network load = sum of active worker actions/sec

FR-3.3: Event Webhook System

- Outbound webhooks for territory flips, mission completions, threat changes
- Enables external integrations (Discord, analytics, etc.)

Phase 4: Production Scaling (Closes Gap G)

FR-4.1: Delta Updates

- Engine emits only changed fields per tick (not full snapshot)
- Client applies delta patch to local state
- Reduces bandwidth from $O(\text{state_size})$ to $O(\text{changes_per_tick})$

FR-4.2: Regional Sharding

- Split engine into territory-actor processes (each territory = independent actor)
- Territory actors hold authoritative state for their region
- Cross-territory missions use a message bus

FR-4.3: Edge Action Batching

- Caddy (or a dedicated edge worker) batches client actions
- Debounce + coalesce within a 200ms window
- Reduces engine ingress load by $\sim 10\times$ at scale

FR-4.4: Protomaps PMTiles Migration

- Replace Esri World Imagery endpoint with self-hosted Protomaps PMTiles
- Single static file, OSM vector tiles, no per-request cost
- CDN-scalable for millions of concurrent tile requests
- One-line source URL change in `mapStyle.sources.satellite`

13. Non-Functional Requirements

13.1 Performance

METRIC	TARGET	CURRENT
Initial load (TTI)	< 3s	~2s (measured)
Tick latency (engine → render)	< 500ms	~2s (2s tick + render)
Concurrent users	1,000,000+	1 (single operative)
Concurrent outposts	100,000+	16
Activity pings/sec	10,000+	~2/tick (1/sec)
Map tile requests	CDN-cached	Direct to Esri
Bundle size	< 500KB gzipped	~350KB (estimated)

13.2 Reliability

- Engine auto-restart on crash (Bun `--hot` provides file-watch restart; need process supervisor for crash recovery)
- Socket.io reconnection (already implemented: `reconnection: true, reconnectionDelay: 800`)
- State recovery: on engine restart, restore last snapshot from SQLite `StateSnapshot` table
- Backup integrity: daily automated bundle refresh to all 3 persistent locations

13.3 Security

- NextAuth.js session validation on all mutating API routes
- Rate limiting: 10 actions/min/operative (configurable)
- Input validation: Zod schemas on all API inputs
- WebSocket auth: session token in socket handshake
- No client-side `z-ai-web-dev-sdk` usage (enforced — server-only)

13.4 Accessibility

- WCAG 2.1 AA compliance
- Keyboard navigation (already partially implemented: 1–8 views, ESC cancel)
- Screen reader support (`sr-only` labels on all icon-only buttons)
- Minimum 44px touch targets (already enforced via shadcn/ui)
- Color contrast: status colors must meet 4.5:1 contrast ratio against dark background

13.5 Browser Support

- Chrome/Edge 120+ (WebGL2 for MapLibre globe)
- Firefox 120+
- Safari 17+ (WebGL2 + socket.io)
- Mobile: iOS Safari 17+, Chrome Android (touch-zoom-rotate enabled)

14. Technical Architecture Evolution

14.1 Current Architecture (MVP)

```
[Browser] ↔socket.io↔ [Engine :3003 (in-memory, single-process)]
    ↓                               ↑
[Next.js :3000] ↔REST proxy↔
    ↓
[SQLite (operative + warlog)]
```

14.2 Target Architecture (Production)



14.3 Migration Path

The architecture evolves in 4 phases (mapping to the functional requirements):

1. **Phase 1** (Dashboard): Frontend-only — no backend changes
2. **Phase 2** (Multi-user): Auth + persistence — NextAuth + StateSnapshot table
3. **Phase 3** (Real telemetry): Action API + worker protocol
4. **Phase 4** (Scaling): Delta updates + sharding + edge batching + PMTiles

Each phase is independently deployable and does not break the previous phase.

15. Phased Roadmap

PHASE	FOCUS	DURATION	KEY DELIVERABLES	SUCCESS METRIC
1	Dashboard & Telemetry	2 weeks	Unit model, unit list, stats dashboard, status colors	Stats view renders 5+ charts; unit list scrolls
2	Multi-User & Persistence	3 weeks	Auth, per-user state, replay, WarLog	10+ concurrent operatives; replay scrubs 24h
3	Real Telemetry	2 weeks	Action API, worker heartbeat, webhooks	1000+ real actions/min ingested
4	Production Scaling	4 weeks	Delta updates, sharding, edge batching, PMTiles	10,000+ concurrent users; <500ms tick latency

15.1 Phase 1 Detailed Tasks

1. Extend `types.ts` with `Unit` interface and status taxonomy
2. Update engine to synthesize per-unit telemetry each tick
3. Build `UnitDetailPanel` component (wireframe SVG + metrics)
4. Build `UnitList` horizontal scrollable component (bottom bar)

- 5. Build `StatsView` with Recharts (5 charts: strength line, territory bar, mission donut, leaderboard table, network area)
- 6. Build `PerformanceView` (per-operative metrics)
- 7. Add status color system to `globals.css` and apply to status dots/badges
- 8. Add `STATS` and `PERF` to NavRail registry
- 9. Lint + browser-verify + commit as new checkpoint

15.2 Phase 2 Detailed Tasks

- 1. Configure NextAuth.js (credential provider, Operative model integration)
- 2. Add session middleware to API routes
- 3. Add `operativeId` to all actions (socket + REST)
- 4. Create `StateSnapshot` Prisma model + migration
- 5. Implement snapshot writer in engine (1/min)
- 6. Build `ReplayView` with timeline scrubber
- 7. Populate `warLog` on mission completion
- 8. Lint + browser-verify + commit as new checkpoint

16. Success Metrics

16.1 Product Metrics

METRIC	TARGET	MEASUREMENT
Daily active operatives	100,000+	Auth sessions per day
Missions per day	1,000,000+	WarLog entries per day
Territory flips per day	500+	Event log flips per day
Average session duration	15+ min	Socket.io connection duration
Replay usage	20% of users	Replay view opens / total users

16.2 Technical Metrics

METRIC	TARGET	MEASUREMENT
Uptime	99.9%	HTTP 200 on <code>/api/state</code>
Tick latency	< 500ms	Engine tick → client render
API error rate	< 0.1%	Non-2xx responses / total
Map tile cache hit rate	> 90%	CDN cache hits / total tile requests
Bundle size	< 500KB gzipped	Production build output

16.3 Backup Integrity Metrics

METRIC	TARGET	MEASUREMENT
Backup freshness	< 24h	Time since last bundle refresh
Restore success rate	100%	Monthly restore drill
Persistent storage availability	3/3 locations	All 3 mounts respond to <code>ls</code>

17. Risks & Mitigations

RISK	PROBABILITY	IMPACT	MITIGATION
Engine crash loses in-memory state	Medium	High	Phase 2 adds snapshot persistence; engine restores from last snapshot on restart
Esri rate-limits tile requests at scale	High	Medium	Phase 4 migrates to self-hosted PMTiles (no rate limit)
MapLibre globe performance degrades with >1000 markers	Medium	High	Clustering already implemented; Phase 1 adds viewport culling for units
LLM API costs scale with users	High	Medium	Cache briefings per-tick (not per-user); fall back to heuristic for common states
Single-process engine bottleneck	High	High	Phase 4 shards by territory; horizontal scaling
Backup storage corruption	Low	Critical	3 independent persistent locations (2× OSS + PolarFS); all verified restorable
Auth integration breaks existing single-operative flow	Medium	Medium	Guest mode preserved; feature flag for auth-required features

18. Appendix

18.1 File Inventory (142 tracked files)

PATH	PURPOSE	LINES
<code>src/components/command/world-map.tsx</code>	3D globe + all map layers	~1090
<code>mini-services/game-engine/index.ts</code>	Authoritative game engine	895
<code>src/lib/types.ts</code>	Shared domain types	257
<code>src/stores/command.ts</code>	Zustand client store	112
<code>src/components/command/command-deck.tsx</code>	Main UI shell	250
<code>src/app/api/ai/briefing/route.ts</code>	LLM tactical briefing	165
<code>src/app/api/state/route.ts</code>	Initial state proxy	19
<code>prisma/schema.prisma</code>	Database schema	35
<code>RESTORE.sh</code>	Backup restore script	171
<code>worklog.md</code>	Development history	600+
<code>.githubhooks/pre-reset</code>	Auto-backup hook	28
<code>.githubhooks/pre-commit</code>	Bundle refresh hook	27

18.2 API Surface

ENDPOINT	METHOD	PURPOSE
<code>/api/state</code>	GET	Initial GameState snapshot (proxy to engine)
<code>/api/ai/briefing</code>	POST	LLM tactical briefing
<code>/api/ai/outpost-briefing</code>	POST	Per-outpost LLM assessment

18.3 Socket Events

EVENT	DIRECTION	PAYLOAD
state	Server → Client	Full GameState (every 2s)
action	Client → Server	ClientAction (launch-mission, place-outpost, upgrade-outpost, request-briefing)
action-result	Server → Client	{ ok, error?, note? }

18.4 Glossary

- **ARIA** — AI Reconnaissance & Intelligence Advisor (the LLM co-pilot)
- **FANG / HAMMER / RESOLUTE** — The three factions
- **FULL outpost** — A full datacenter-grade node (high compute/health)
- **TACTICAL outpost** — A browser-based worker node (lower compute)
- **SAFEHOUSE** — A user-placed fortified node
- **Sol** — Simulated "day" counter (game time)
- **VOTC** — Network-universal currency
- **Tick** — Engine game-loop cycle (2 seconds)
- **Territory** — A named geographic region factions fight to control
- **Activity Ping** — A transient live-action point (visualizes planet-scale activity)

19. Document Control

FIELD	VALUE
Author	Z.ai Code
Checkpoint	debd909
Status	Final
Next Review	After Phase 1 completion
Distribution	Project team